In [58]:

```
import warnings
warnings.filterwarnings('ignore')
```

```
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarnin
g: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future versio
n. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datet
ime.UTC).
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarnin
g: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future versio
n. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datet
ime.UTC).
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarnin
g: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future versio
n. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datet
ime.UTC).
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarnin
g: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future versio
n. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datet
ime.UTC).
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarnin
g: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future versio
n. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datet
ime.UTC).
  return datetime.utcnow().replace(tzinfo=utc)
```

Q1: Load the data.

In [59]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split

heart = pd.read_csv('Heart.csv', index_col=0)
print(heart.head())

df = pd.DataFrame(heart)
```

```
   Age  Sex     ChestPain  RestBP  Chol  Fbs  RestECG  MaxHR  ExAng  Oldpeak  \
1   63    1       typical     145   233    1        2    150      0      2.3   
2   67    1  asymptomatic     160   286    0        2    108      1      1.5   
3   67    1  asymptomatic     120   229    0        2    129      1      2.6   
4   37    1    nonanginal     130   250    0        0    187      0      3.5   
5   41    0    nontypical     130   204    0        2    172      0      1.4   

   Slope   Ca        Thal  AHD  
1      3  0.0       fixed   No  
2      2  3.0      normal  Yes  
3      2  2.0  reversable  Yes  
4      3  0.0      normal   No  
5      1  0.0      normal   No  
```

Q1: How many variables in the dataset? What are they? Are they quantitative or qualitative variables? Is there any missing value?

In [60]:

```
print("The number of variables are:", df.shape[1]) #gets number of total variables

print("The variable names are:" , list(df.columns))

print(df.info())

# —— Based on data types
quantitative_vars = df.select_dtypes(include=["int64", "float64"]).columns.tolist()
qualitative_vars  = df.select_dtypes(include=["object"]).columns.tolist()

# —— Print results
print(f"Total variables: {df.shape[1]}")
print()
print(f"Quantitative variables ({len(quantitative_vars)}): {quantitative_vars}")
print()
print(f"Qualitative variables ({len(qualitative_vars)}): {qualitative_vars}")


# —— Missing value check
print("Missing values per variable:")
print(df.isnull().sum())
print()
print(f"Total missing values: {df.isnull().sum().sum()}")
```

```
The number of variables are: 14
The variable names are: ['Age', 'Sex', 'ChestPain', 'RestBP', 'Chol', 'Fbs', 'RestECG',
'MaxHR', 'ExAng', 'Oldpeak', 'Slope', 'Ca', 'Thal', 'AHD']
<class 'pandas.core.frame.DataFrame'>
Index: 303 entries, 1 to 303
Data columns (total 14 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Age        303 non-null    int64
 1   Sex        303 non-null    int64
 2   ChestPain  303 non-null    object
 3   RestBP     303 non-null    int64
 4   Chol       303 non-null    int64
 5   Fbs        303 non-null    int64
 6   RestECG    303 non-null    int64
 7   MaxHR      303 non-null    int64
 8   ExAng      303 non-null    int64
 9   Oldpeak    303 non-null    float64
 10  Slope      303 non-null    int64
 11  Ca         299 non-null    float64
 12  Thal       301 non-null    object
 13  AHD        303 non-null    object
dtypes: float64(2), int64(9), object(3)
memory usage: 35.5+ KB
None
Total variables: 14

Quantitative variables (11): ['Age', 'Sex', 'RestBP', 'Chol', 'Fbs', 'RestECG', 'MaxHR',
'ExAng', 'Oldpeak', 'Slope', 'Ca']
```

```
Qualitative variables (3): ['ChestPain', 'Thal', 'AHD']
Missing values per variable:
Age          0
Sex          0
ChestPain    0
RestBP       0
Chol         0
Fbs          0
RestECG      0
MaxHR        0
ExAng        0
Oldpeak      0
Slope        0
Ca           4
Thal         2
AHD          0
dtype: int64

Total missing values: 6
```

Q2. Impute the missing value using mean value for quantitative variable and majority class for qualitative variable.

In [61]:

```
# ── Before
print("Before imputation:")
print(df.isnull().sum())

# ── Impute Ca (quantitative) with mean
df["Ca"]   = df["Ca"].fillna(df["Ca"].mean())

# ── Impute Thal (qualitative) with majority class
df["Thal"] = df["Thal"].fillna(df["Thal"].mode()[0])

# ── After
print("\nAfter imputation:")
print(df.isnull().sum())
```

```
Before imputation:
Age          0
Sex          0
ChestPain    0
RestBP       0
Chol         0
Fbs          0
RestECG      0
MaxHR        0
ExAng        0
Oldpeak      0
Slope        0
Ca           4
Thal         2
AHD          0
dtype: int64

After imputation:
Age          0
Sex          0
```

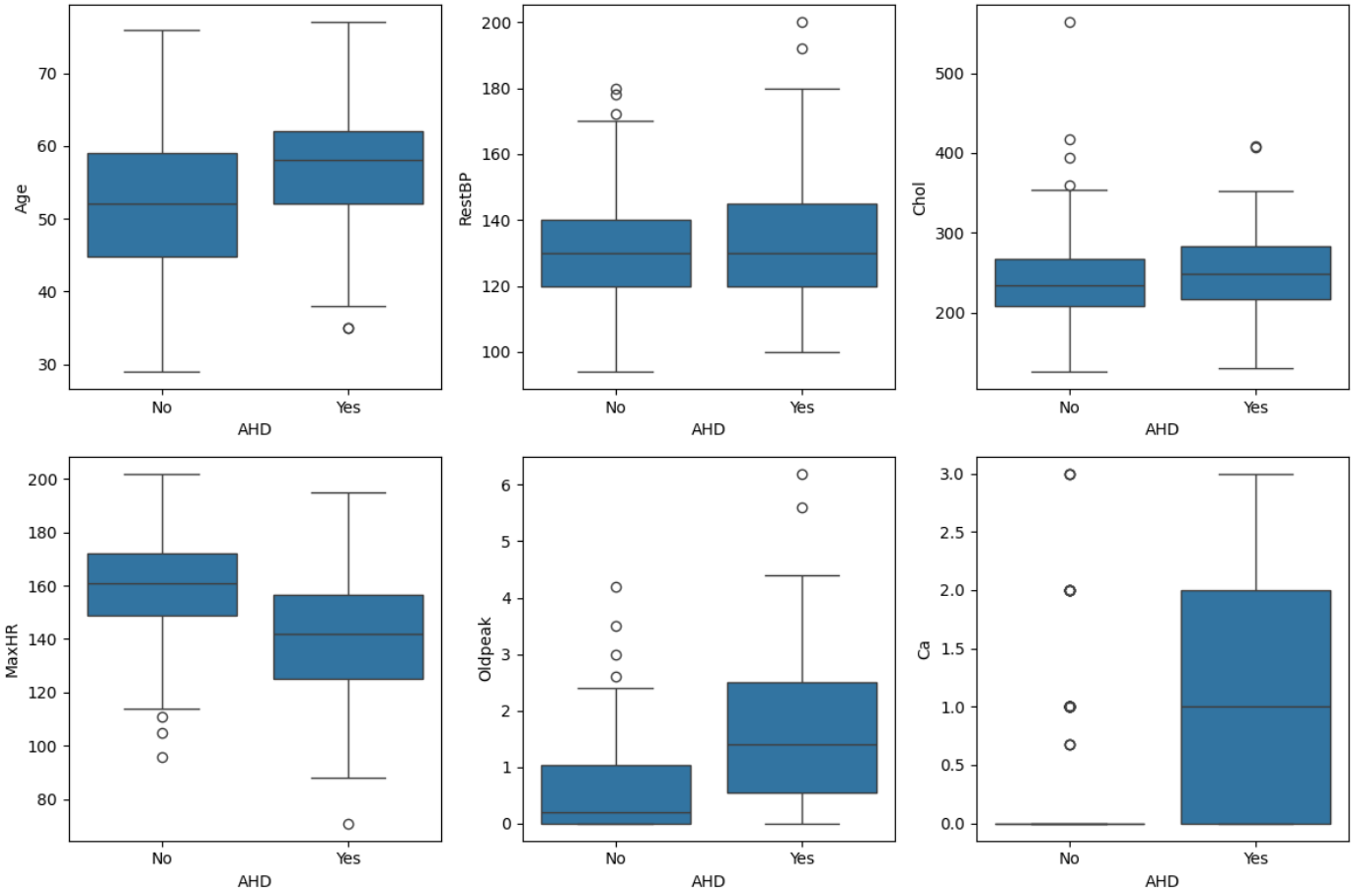
```
ChestPain    0
RestBP       0
Chol         0
Fbs          0
RestECG      0
MaxHR        0
ExAng        0
Oldpeak      0
Slope        0
Ca           0
Thal         0
AHD          0
dtype: int64
```

Q3. Data visualization.

a) Please use the boxplots to visualize the relationship of quantitative predictor with outcome (AHD).

In [62]:

```
# a) Boxplots
plt.figure(figsize=(12, 8))

plt.subplot(2, 3, 1)
sns.boxplot(x='AHD', y='Age', data=df)

plt.subplot(2, 3, 2)
sns.boxplot(x='AHD', y='RestBP', data=df)

plt.subplot(2, 3, 3)
sns.boxplot(x='AHD', y='Chol', data=df)

plt.subplot(2, 3, 4)
sns.boxplot(x='AHD', y='MaxHR', data=df)

plt.subplot(2, 3, 5)
sns.boxplot(x='AHD', y='Oldpeak', data=df)

plt.subplot(2, 3, 6)
sns.boxplot(x='AHD', y='Ca', data=df)

plt.tight_layout()
plt.show()
```

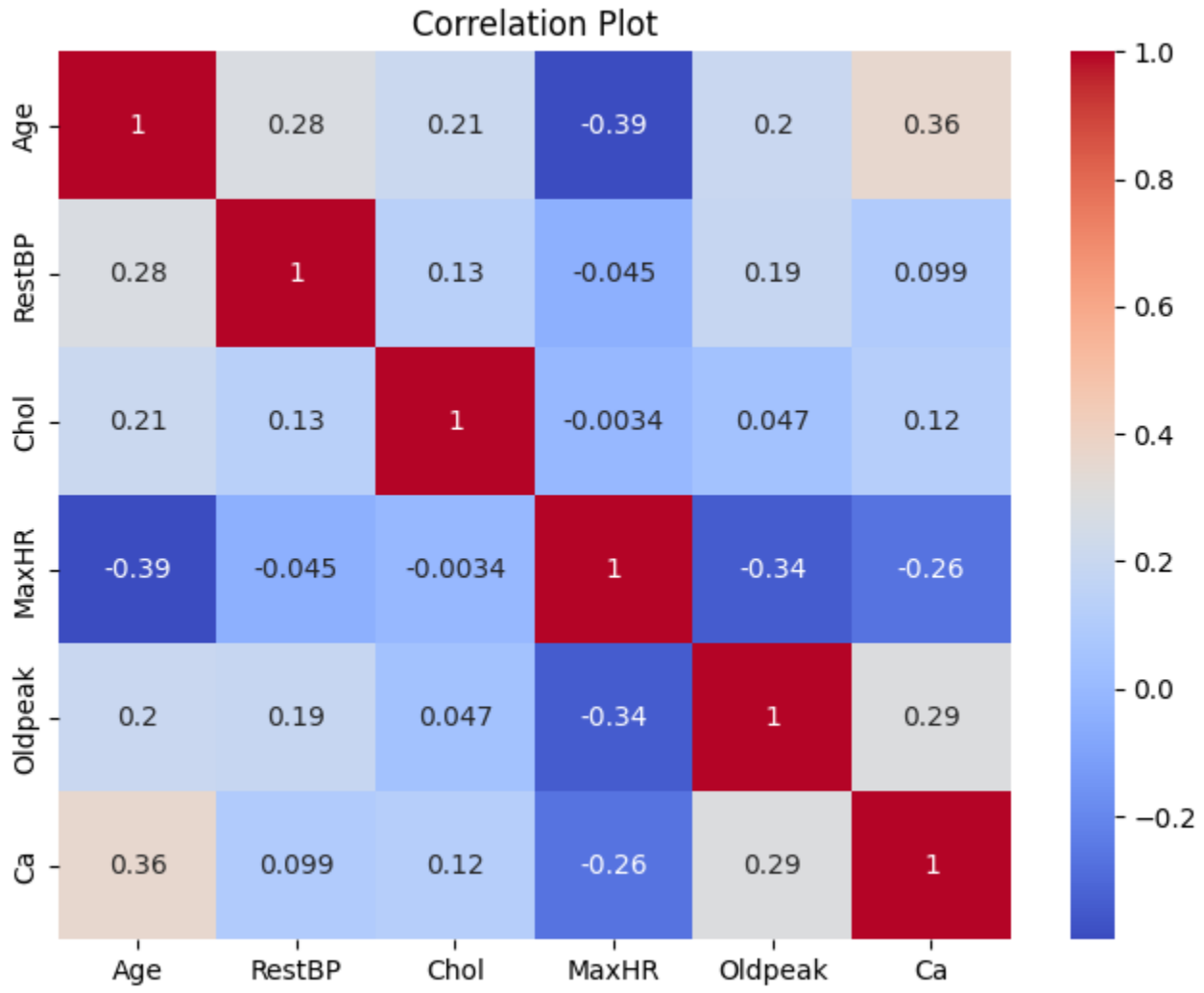


Patients with heart disease (AHD=Yes) tend to be older, have lower MaxHR, higher Oldpeak, and more blocked vessels (Ca), making these three the strongest predictors. RestBP shows a slight difference while Chol shows almost no difference between the two groups.

b) Please use the corrplots to visualize the correlations between quantitative variables.

In [63]:

```
import matplotlib.pyplot as plt
import seaborn as sns

quant_vars = ["Age", "RestBP", "Chol", "MaxHR", "Oldpeak", "Ca"]

plt.figure(figsize=(8, 6))
sns.heatmap(df[quant_vars].corr(), annot=True, cmap="coolwarm")
plt.title("Correlation Plot")
plt.show()
```



- **Age & MaxHR** have the strongest relationship (-0.39) — older patients have lower maximum heart rate
- **Age** is positively related to both **Ca** (0.36) and **RestBP** (0.28) — older patients tend to have more blocked vessels and higher blood pressure
- **MaxHR & Oldpeak** (-0.34) and **Ca & Oldpeak** (0.29) suggest that poor cardiac performance is linked to higher ST depression
- **Cholesterol** (Chol) shows almost no correlation with any variable (-0.003 to 0.12), making it the least related variable in the dataset

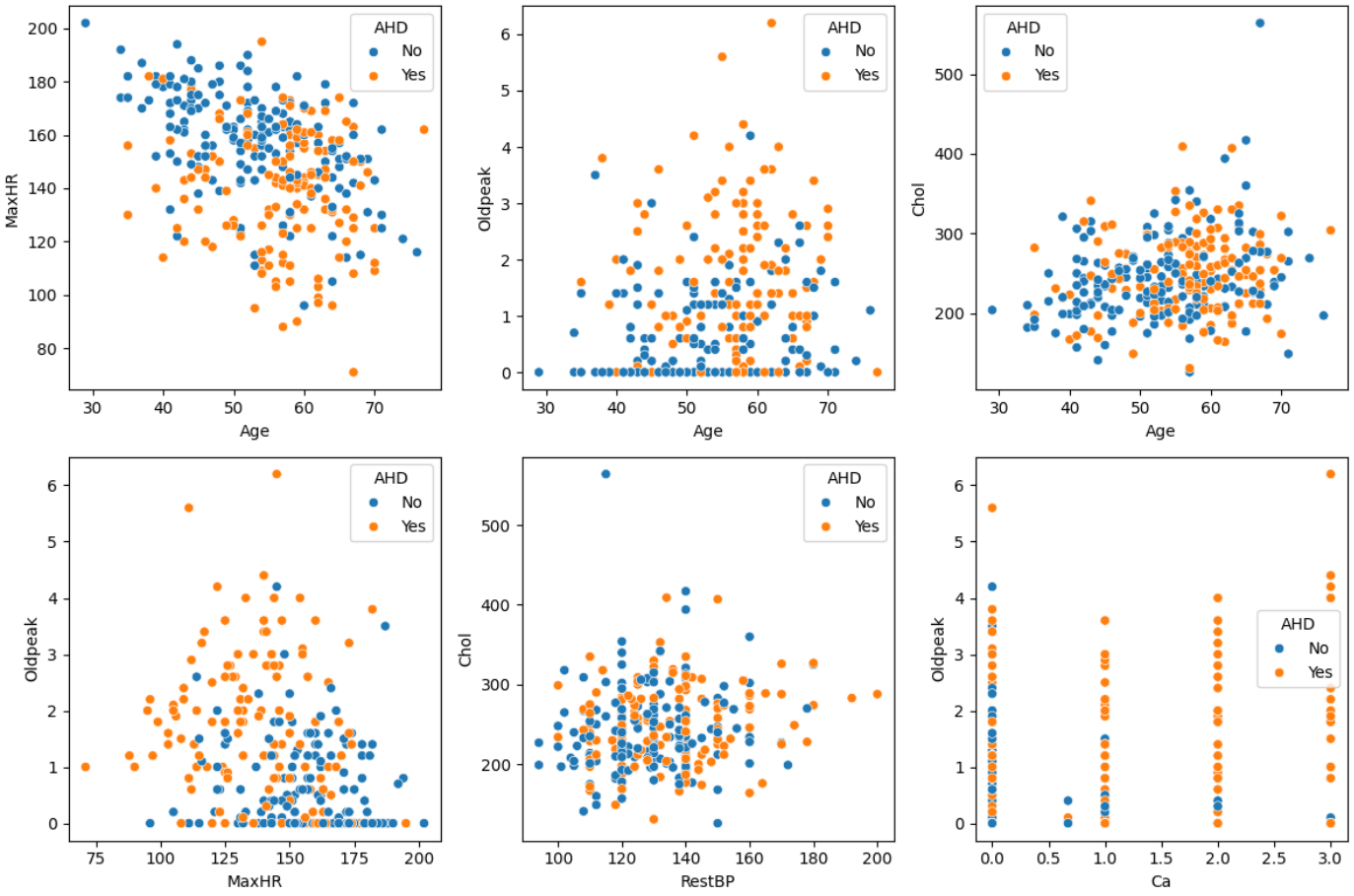
Overall, no strong correlations (above 0.70) exist between any two variables, meaning multicollinearity is not a concern in this dataset

c) Please use the scatterplots to find the patterns in predictors that distinguish AHD=Yes and AHD=No patients.

In [64]:

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(12, 8))

plt.subplot(2, 3, 1)
sns.scatterplot(x='Age', y='MaxHR', hue='AHD', data=df)
```

```
plt.subplot(2, 3, 2)
sns.scatterplot(x='Age', y='Oldpeak', hue='AHD', data=df)

plt.subplot(2, 3, 3)
sns.scatterplot(x='Age', y='Chol', hue='AHD', data=df)

plt.subplot(2, 3, 4)
sns.scatterplot(x='MaxHR', y='Oldpeak', hue='AHD', data=df)

plt.subplot(2, 3, 5)
sns.scatterplot(x='RestBP', y='Chol', hue='AHD', data=df)

plt.subplot(2, 3, 6)
sns.scatterplot(x='Ca', y='Oldpeak', hue='AHD', data=df)

plt.tight_layout()
plt.show()
```



- **Age vs MaxHR** — As patients get older, their max heart rate drops. Orange dots (sick) sit at the bottom and blue dots (healthy) sit at the top, showing a clear separation between the two groups
- **MaxHR vs Oldpeak** — This is the clearest plot. Sick patients (orange) cluster at low MaxHR and high Oldpeak, while healthy patients (blue) cluster at high MaxHR and low Oldpeak — almost no mixing between the two groups
- **Ca vs Oldpeak**— Healthy patients (blue) are mostly stuck at Ca=0, while sick patients (orange) spread across Ca=1, 2 and 3, showing that blocked vessels are a strong sign of heart disease
- **Age vs Chol** — Blue and orange dots are completely mixed together with no pattern, meaning cholesterol cannot help us tell sick patients from healthy ones RestBP vs Chol — Both groups are

scattered randomly with no separation at all, confirming that blood pressure and cholesterol together are useless in identifying heart disease

- **Overall** — Patients with low MaxHR, high Oldpeak and high Ca are very likely to have heart disease, while patients with high MaxHR, low Oldpeak and Ca near 0 are likely healthy

Q4. Please separate the dataset into training and testing data with ratio of 3:1.

In [65]:

```
# Convert categorical to numerical
heart_encoded = pd.get_dummies(df, columns=["ChestPain", "Thal"], drop_first=True)
heart_encoded["AHD"] = heart_encoded["AHD"].map({"No": 0, "Yes": 1})

# Separate features and target
X = heart_encoded.drop(columns=["AHD"])
y = heart_encoded["AHD"]

# Split 75% train, 25% test (3:1 ratio)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=4

print(f"Total samples : {len(df)}")
print(f"Training size : {len(X_train)}")
print(f"Testing size  : {len(X_test)}")
heart_encoded
```

```
Total samples : 303
Training size : 227
Testing size  : 76
```

Out[65]:

	Age	Sex	RestBP	Chol	Fbs	RestECG	MaxHR	ExAng	Oldpeak	Slope	Ca	AHD	ChestI
1	63	1	145	233	1	2	150	0	2.3	3	0.000000	0	
2	67	1	160	286	0	2	108	1	1.5	2	3.000000	1	
3	67	1	120	229	0	2	129	1	2.6	2	2.000000	1	
4	37	1	130	250	0	0	187	0	3.5	3	0.000000	0	
5	41	0	130	204	0	2	172	0	1.4	1	0.000000	0	
...	...	...	...	...	...	...	...	...	...	...	...	...	
299	45	1	110	264	0	0	132	0	1.2	2	0.000000	1	
300	68	1	144	193	1	0	141	0	3.4	2	2.000000	1	
301	57	1	130	131	0	0	115	1	1.2	2	1.000000	1	
302	57	0	130	236	0	2	174	0	0.0	2	1.000000	1	
303	38	1	138	175	0	0	173	0	0.0	1	0.672241	0	

303 rows × 17 columns

Q5. LOGISTIC REGRESSION

In [66]:

```
from sklearn.linear_model import LogisticRegression

# Fit logistic regression on training data
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

print("Logistic Regression model fitted successfully!")
print(f"Number of features used: {X_train.shape[1]}")
```

```
Logistic Regression model fitted successfully!
Number of features used: 16
```

a) Is there any significant predictor?

In [67]:

```
# Get coefficients for each predictor
coef_df = pd.DataFrame({
    "Predictor"  : X_train.columns,
    "Coefficient": model.coef_[0].round(4)
}).sort_values("Coefficient", key=abs, ascending=False)

print("Coefficients of each predictor:")
print(coef_df.to_string(index=False))

print("\nSignificant predictors (|coef| > 0.5):")
print(coef_df[abs(coef_df["Coefficient"]) > 0.5])
```

```
Coefficients of each predictor:
           Predictor  Coefficient
                  Ca       1.3416
                 Sex       1.2616
ChestPain_nonanginal      -1.1892
     Thal_reversable       0.9565
   ChestPain_typical      -0.9180
               ExAng       0.7872
ChestPain_nontypical      -0.6831
               Slope       0.4417
             Oldpeak       0.2988
             RestECG       0.1354
         Thal_normal      -0.0851
                 Fbs      -0.0531
               MaxHR      -0.0197
              RestBP       0.0177
                Chol       0.0066
                 Age       0.0011

Significant predictors (|coef| > 0.5):
               Predictor  Coefficient
10                    Ca       1.3416
1                    Sex       1.2616
11  ChestPain_nonanginal      -1.1892
15       Thal_reversable       0.9565
13     ChestPain_typical      -0.9180
7                  ExAng       0.7872
12  ChestPain_nontypical      -0.6831
```

Yes, there are 7 significant predictors

b) Interpret the effects of significant predictors.

1. Increase the risk of heart disease (+):

- **Ca (+1.35)** — more blocked vessels = higher risk
- **Sex (+1.26)** — being male = higher risk
- **Thal_reversable (+0.97)** — reversable thalassemia = higher risk
- **ExAng (+0.79)** — exercise-induced angina = higher risk

2. Decrease the risk of heart disease (-):

- **ChestPain_nonanginal (-1.18)** — non-anginal chest pain = lower risk
- **ChestPain_typical (-0.91)** — typical chest pain = lower risk
- **ChestPain_nontypical (-0.68)** — non-typical chest pain = lower risk

3. Not significant (coefficient near 0):

- **Age, Chol, RestBP, MaxHR** — have very small coefficients, meaning they have little to no effect on predicting heart disease in this model

Key takeaway:

- A patient who is male + has blocked vessels + exercise angina + reversable thalassemia is at the highest risk of heart disease
- A patient with non-anginal chest pain is at the lowest risk

c) How good this model fits the data

I. Confusion matrix

II. Accuracy

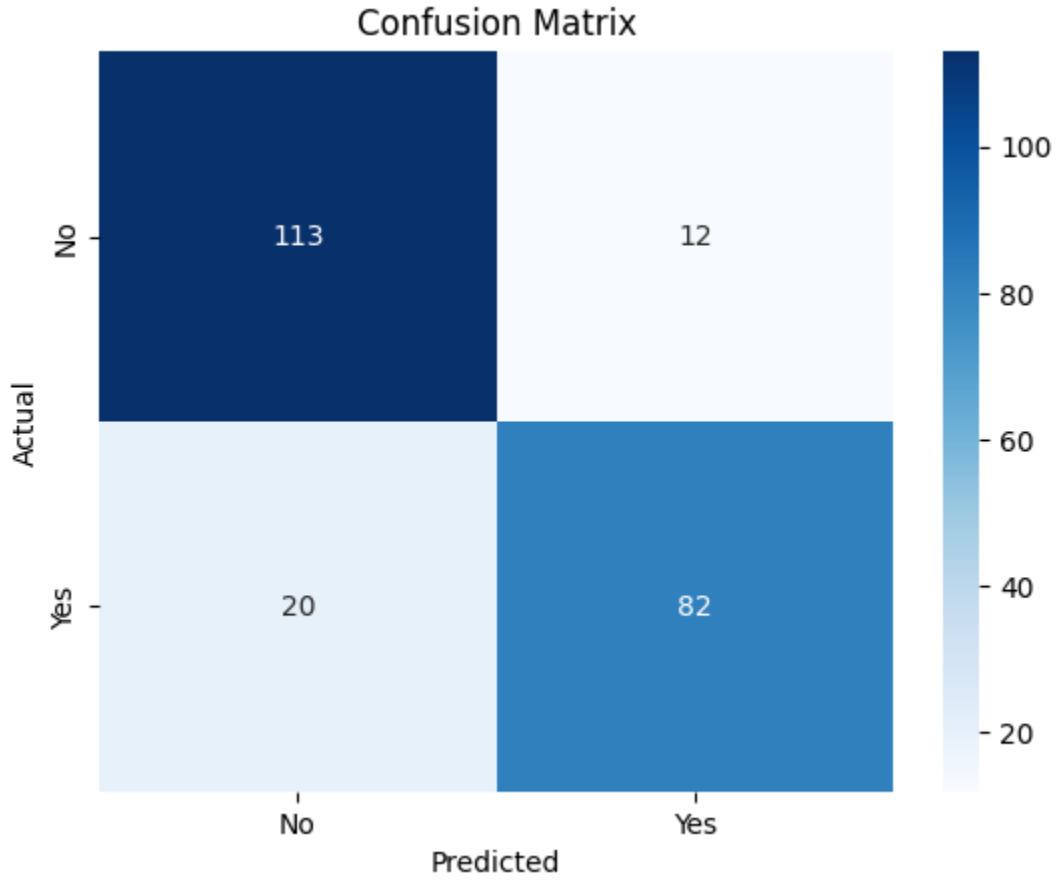
III. ROC curve and AUC

In [68]:

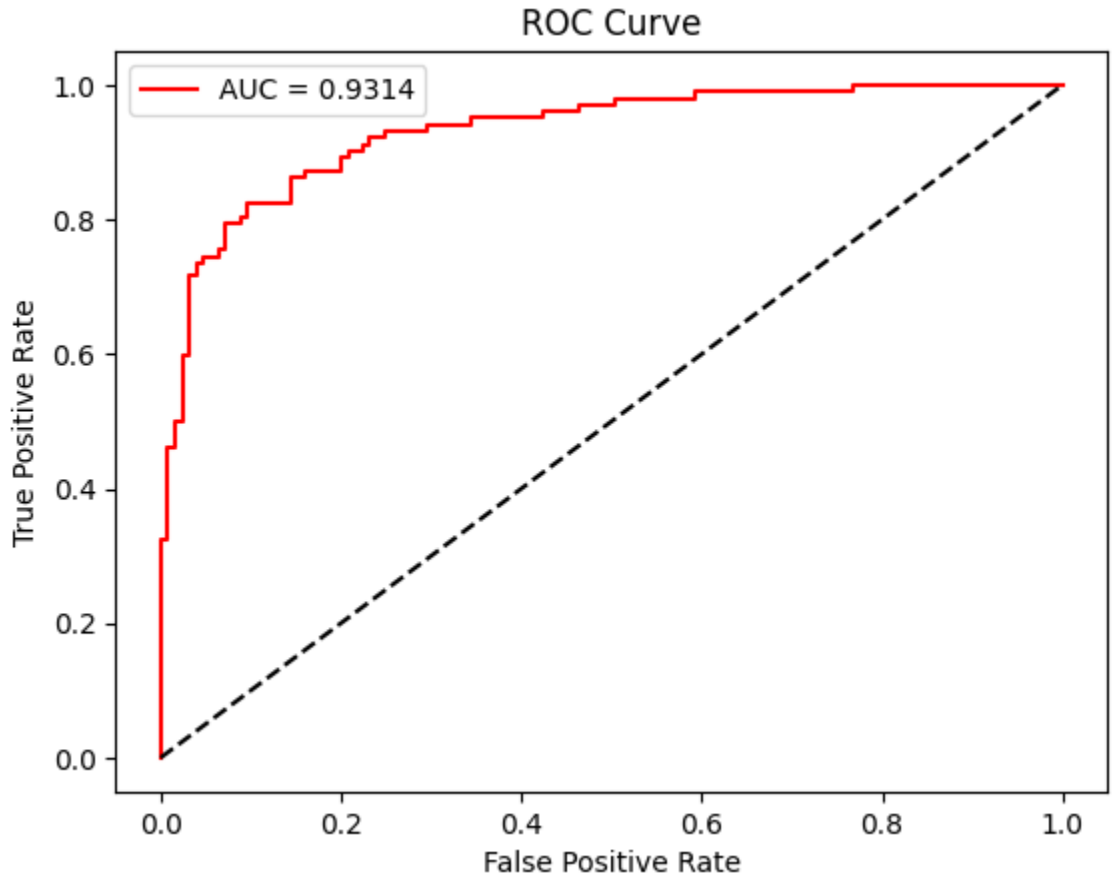
```
from sklearn.metrics import confusion_matrix, accuracy_score, roc_auc_score, roc_curve
import matplotlib.pyplot as plt
import seaborn as sns

# ── Training predictions ──────────────────────────────────────
y_train_pred = model.predict(X_train)
y_train_prob = model.predict_proba(X_train)[:, 1]

# I. Confusion Matrix
print("=== Training Performance ===")
cm = confusion_matrix(y_train, y_train_pred)
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=["No","Yes"], yticklabels=["No","Yes"])
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

```python
# II. Accuracy
acc = accuracy_score(y_train, y_train_pred)
print(f"Accuracy : {acc:.4f}")

# III. ROC Curve and AUC
auc = roc_auc_score(y_train, y_train_prob)
fpr, tpr, _ = roc_curve(y_train, y_train_prob)
plt.plot(fpr, tpr, color="red", label=f"AUC = {auc:.4f}")
plt.plot([0,1], [0,1], "k--")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.show()

print(f"AUC      : {auc:.4f}")
```

```
=== Training Performance ===
```




```
Accuracy : 0.8590
```




```
AUC      : 0.9314
```

Confusion Matrix:

- 113 patients correctly predicted as No (True Negative) ✅
- 82 patients correctly predicted as Yes (True Positive) ✅
- 12 patients actually No but predicted as Yes (False Positive) ❌
- 20 patients actually Yes but predicted as No (False Negative) ❌

Accuracy = 0.8590

- Model correctly predicted 85.90% of training patients
- Out of 227 training patients, (113+82) = 195 were correct and only 32 were wrong

ROC Curve & AUC = 0.9314

- The red curve is far above the dashed line — meaning the model is much better than random guessing
- AUC of 0.93 is considered excellent (closer to 1.0 = perfect model)
- The curve rises steeply towards the top-left corner which is the ideal position

Overall conclusion:

- The model fits the training data very well
- AUC 0.93 tells us the model has excellent ability to distinguish between heart disease and healthy patients
- The only concern is 20 False Negatives — these are sick patients predicted as healthy, which is the most dangerous type of error in medical diagnosis

d) Evaluate the model on testing data

I. Confusion matrix

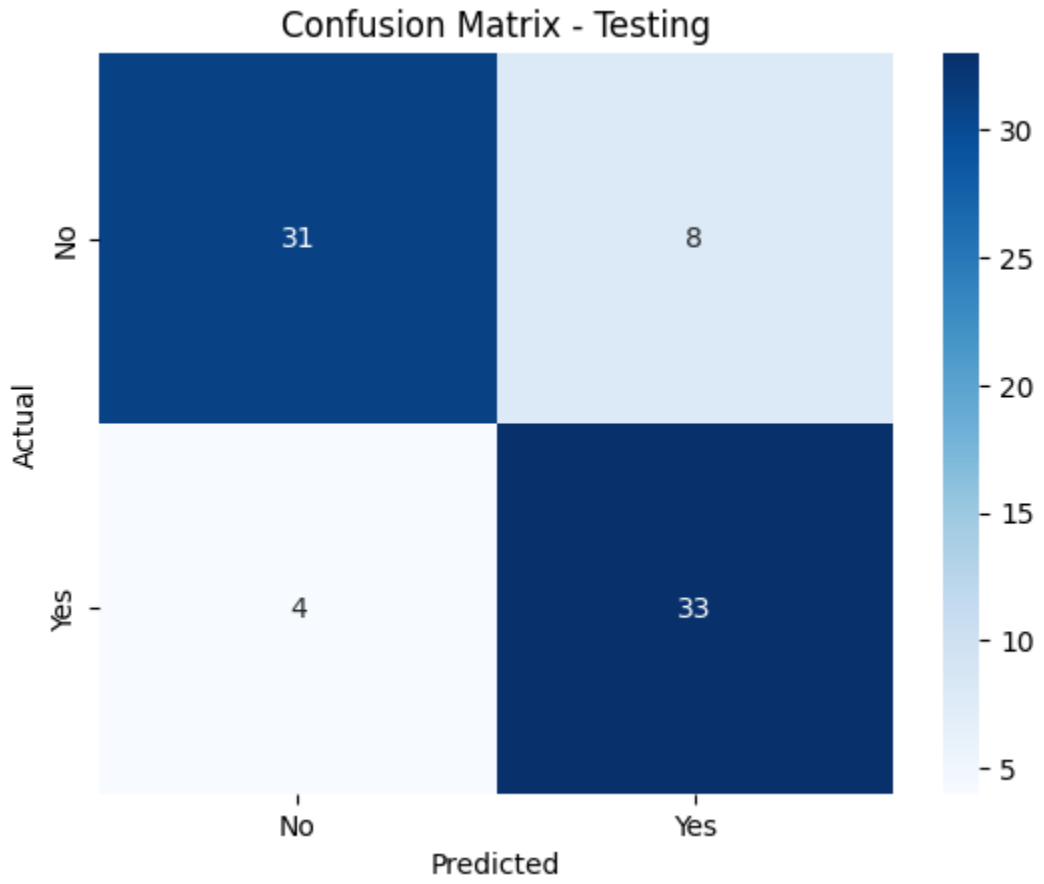
II. Accuracy

III. ROC curve and AUC

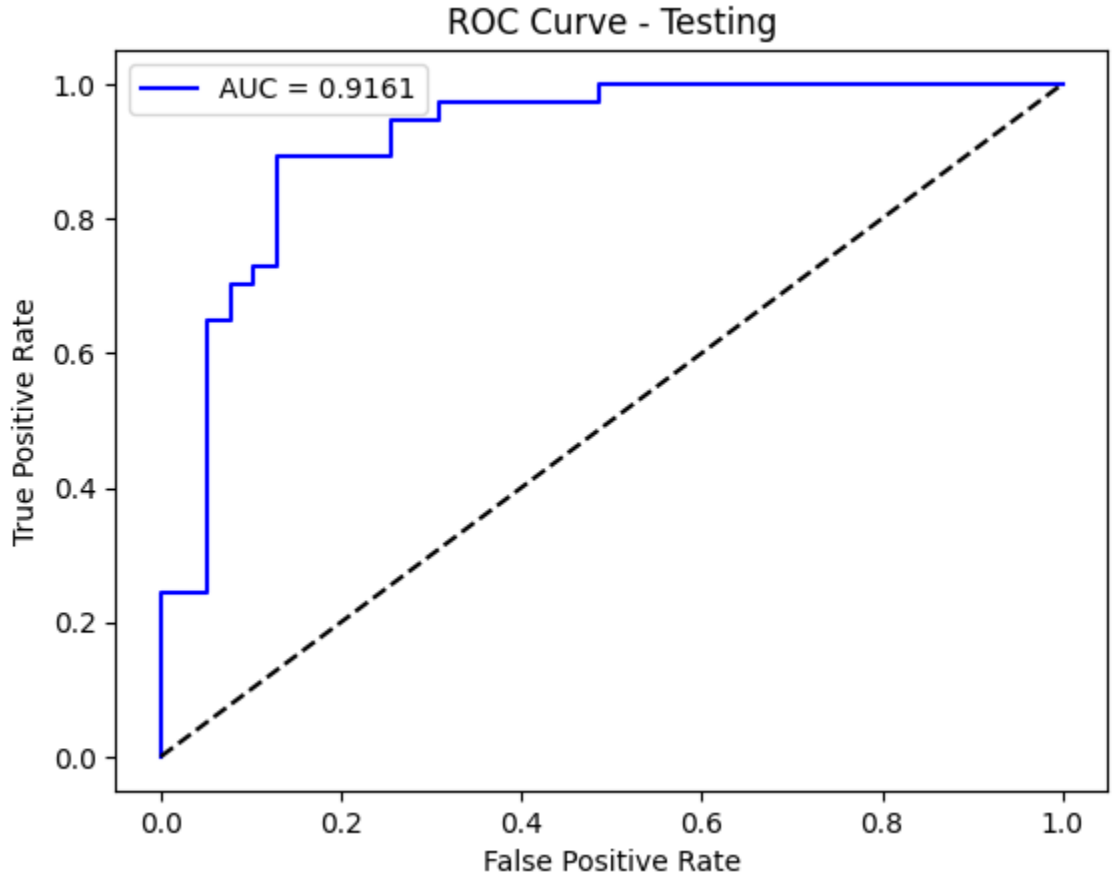
In [69]:

```
# ── Testing predictions ──────────────────────────────────────────
y_test_pred = model.predict(X_test)
y_test_prob = model.predict_proba(X_test)[:, 1]

# I. Confusion Matrix
print("=== Testing Performance ===")
cm = confusion_matrix(y_test, y_test_pred)
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=["No","Yes"], yticklabels=["No","Yes"])
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix - Testing")
plt.show()

# II. Accuracy
acc = accuracy_score(y_test, y_test_pred)
print(f"Accuracy : {acc:.4f}")

# III. ROC Curve and AUC
auc = roc_auc_score(y_test, y_test_prob)
fpr, tpr, _ = roc_curve(y_test, y_test_prob)
plt.plot(fpr, tpr, color="blue", label=f"AUC = {auc:.4f}")
plt.plot([0,1], [0,1], "k--")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Testing")
plt.legend()
plt.show()

print(f"AUC      : {auc:.4f}")
```

```
=== Testing Performance ===
```




```
Accuracy : 0.8421
```




```
AUC      : 0.9161
```

Confusion Matrix:

- 31 patients correctly predicted as No (True Negative) ✅
- 33 patients correctly predicted as Yes (True Positive) ✅
- 8 patients actually No but predicted as Yes (False Positive) ❌
- 4 patients actually Yes but predicted as No (False Negative) ❌

Accuracy = 0.8421

- Model correctly predicted 84.21% of testing patients
- Out of 76 testing patients, (31+33) = 64 were correct and only 12 were wrong

ROC Curve & AUC = 0.9161

- The blue curve rises steeply towards the top-left corner — sign of a good model
- AUC of 0.91 is considered excellent
- Model is far above the dashed line meaning much better than random guessing

e) Use the feature selection algorithm to find optimal feature set. How the model perform on training and testing data?

In [70]:

```
from sklearn.feature_selection import RFE

# Step 1: Select best 8 features
rfe = RFE(LogisticRegression(max_iter=1000), n_features_to_select=8)
rfe.fit(X_train, y_train)

selected = X_train.columns[rfe.support_].tolist()
print("Selected Features:", selected)

# Step 2: Train new model with selected features only
model_rfe = LogisticRegression(max_iter=1000)
model_rfe.fit(X_train[selected], y_train)

# Step 3: Training performance
y_train_pred = model_rfe.predict(X_train[selected])
y_train_prob = model_rfe.predict_proba(X_train[selected])[:, 1]

print("\n=== Training Performance ===")
print("Confusion Matrix:\n", confusion_matrix(y_train, y_train_pred))
print("Accuracy :", accuracy_score(y_train, y_train_pred))
print("AUC      :", roc_auc_score(y_train, y_train_prob).round(4))

# Step 4: Testing performance
y_test_pred = model_rfe.predict(X_test[selected])
y_test_prob = model_rfe.predict_proba(X_test[selected])[:, 1]

print("\n=== Testing Performance ===")
print("Confusion Matrix:\n", confusion_matrix(y_test, y_test_pred))
print("Accuracy :", accuracy_score(y_test, y_test_pred))
print("AUC      :", roc_auc_score(y_test, y_test_prob).round(4))
```

```
Selected Features: ['Sex', 'ExAng', 'Slope', 'Ca', 'ChestPain_nonanginal', 'ChestPain_no
ntypical', 'ChestPain_typical', 'Thal_reversable']

=== Training Performance ===
Confusion Matrix:
```

```
 [[110  15]
 [ 20  82]]
Accuracy : 0.8458149779735683
AUC      : 0.9166

=== Testing Performance ===
Confusion Matrix:
 [[34  5]
 [ 6 31]]
Accuracy : 0.8552631578947368
AUC      : 0.904
```

f) Compare the models with and without feature selection, what you observe?

In [71]:

```
print("=== Model Comparison ===")
print(f"{'Metric':<20} {'Full Model':>12} {'RFE Model':>12}")
print("-" * 45)
print(f"{'Train Accuracy':<20} {0.8590:>12} {0.8458:>12}")
print(f"{'Test Accuracy':<20} {0.8421:>12} {0.8553:>12}")
print(f"{'Train AUC':<20} {0.9314:>12} {0.9166:>12}")
print(f"{'Test AUC':<20} {0.9161:>12} {0.9040:>12}")
print(f"{'No. of Features':<20} {16:>12} {8:>12}")
```

```
=== Model Comparison ===
Metric                 Full Model    RFE Model
---------------------------------------------
Train Accuracy              0.859       0.8458
Test Accuracy              0.8421       0.8553
Train AUC                  0.9314       0.9166
Test AUC                   0.9161        0.904
No. of Features                16            8
```

Observations:

- Full model has slightly higher training accuracy (0.8590) but that is because it uses all 16 features including weak ones
- RFE model has slightly higher testing accuracy (0.8553) using only 8 features — meaning it generalizes better to new data
- Both models have similar AUC (~0.91-0.93) — both are excellent at distinguishing heart disease patients
- RFE model uses half the features (8 vs 16) but gives same or better performance
- **Conclusion** — RFE model is the better choice because it is simpler, faster and equally accurate ✅

6. CART model.

a) Please fit a CART model on training data to predict AHD. What is the tree size? Which predictors are used?

In [72]:

```
from sklearn.tree import DecisionTreeClassifier

# Fit CART model
cart = DecisionTreeClassifier(random_state=42)
```

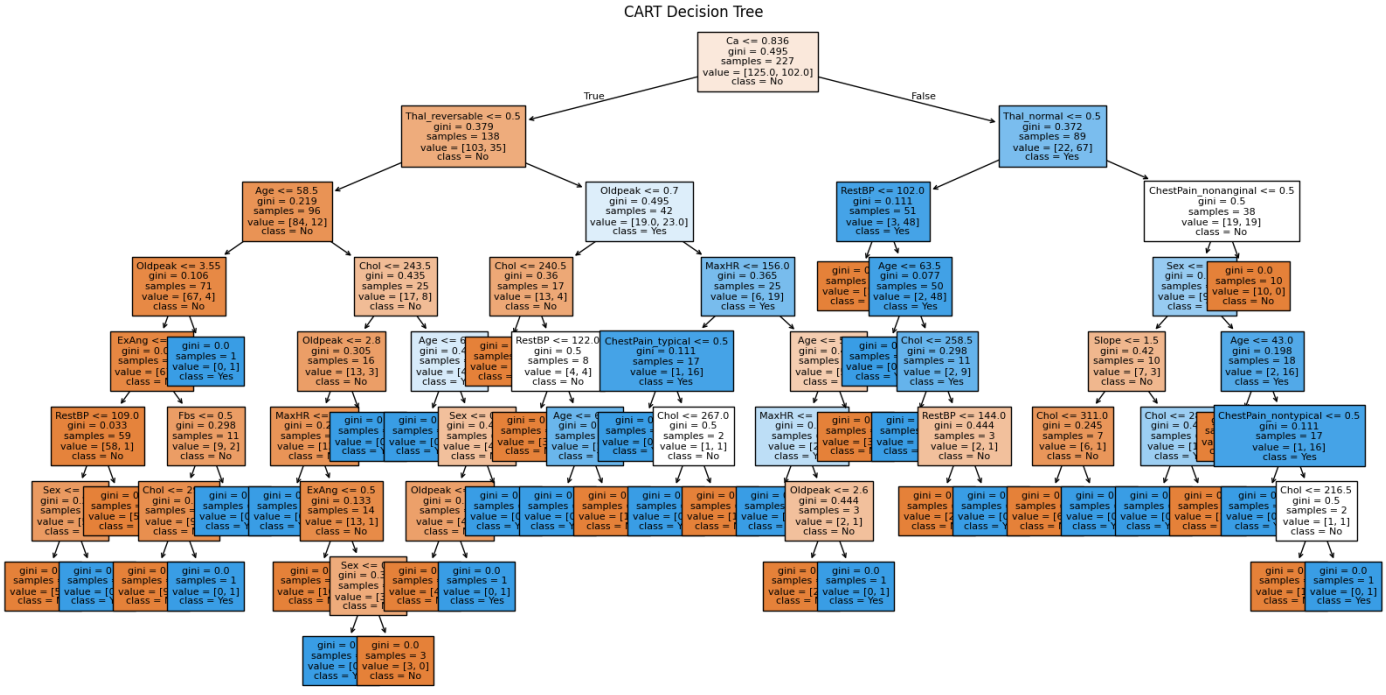
```
cart.fit(X_train, y_train)

# Tree size
print("Tree Size (nodes):", cart.tree_.node_count)
print("Tree Depth        :", cart.get_depth())

# Predictors used
features_used = X_train.columns[cart.feature_importances_ > 0].tolist()
print("Predictors Used   :", features_used)
```

```
Tree Size (nodes): 81
Tree Depth        : 8
Predictors Used   : ['Age', 'Sex', 'RestBP', 'Chol', 'Fbs', 'MaxHR', 'ExAng', 'Oldpeak',
'Slope', 'Ca', 'ChestPain_nonanginal', 'ChestPain_nontypical', 'ChestPain_typical', 'Tha
l_normal', 'Thal_reversable']
```

b) Plot the tree.

In [73]:

```
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

plt.figure(figsize=(20, 10))
plot_tree(cart,
          feature_names=X_train.columns,
          class_names=["No", "Yes"],
          filled=True,
          fontsize=8)
plt.title("CART Decision Tree")
plt.show()
```



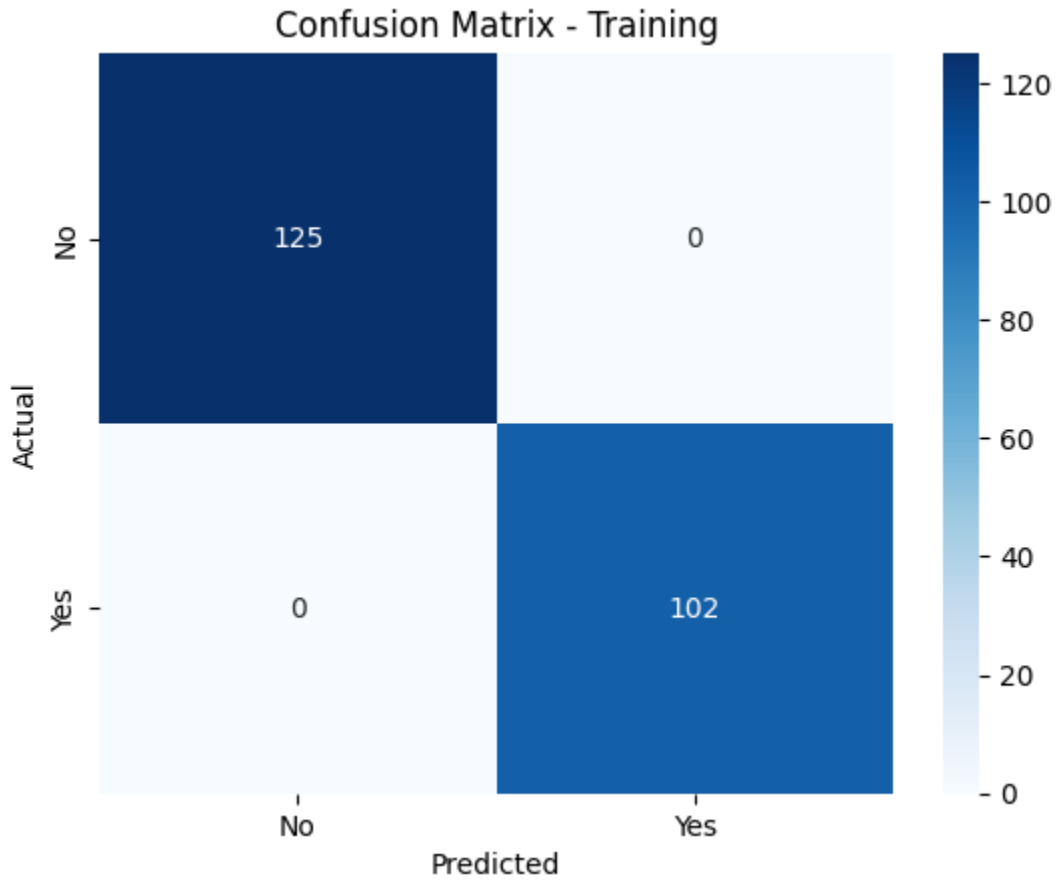
c) Evaluate the prediction accuracy on training and testing data.

In [74]:

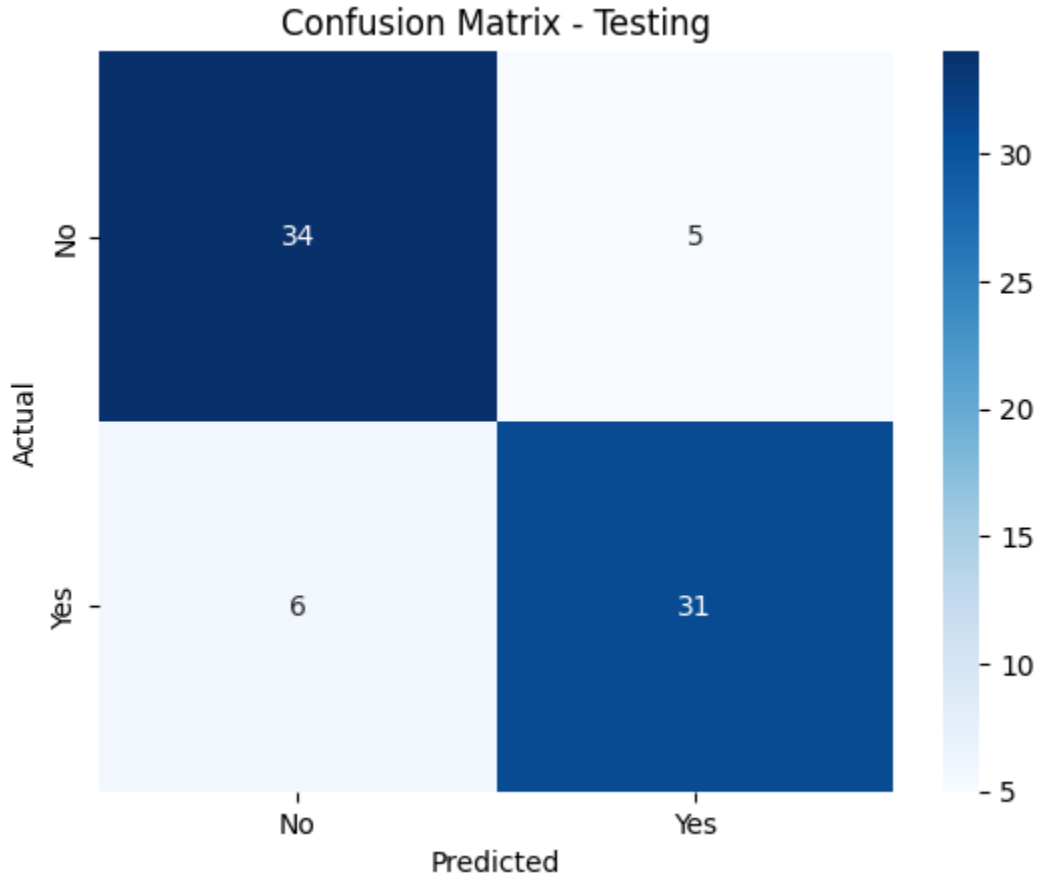
```
# I. Confusion Matrix
print("=== Training Performance ===")
y_train_pred = cart.predict(X_train)
```

```
cm = confusion_matrix(y_train, y_train_pred)
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=["No","Yes"], yticklabels=["No","Yes"])
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix - Training")
plt.show()


print("=== Testing Performance ===")
y_train_pred = cart.predict(X_train)
cm = confusion_matrix(y_test, y_test_pred)
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=["No","Yes"], yticklabels=["No","Yes"])
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix - Testing")
plt.show()

# Training performance
# y_train_pred = cart.predict(X_train)
# print("=== Training Performance ===")
# print("Confusion Matrix:\n", confusion_matrix(y_train, y_train_pred))
# print("Accuracy :", accuracy_score(y_train, y_train_pred))

# Testing performance
# y_test_pred = cart.predict(X_test)
# print("\n=== Testing Performance ===")
# print("Confusion Matrix:\n", confusion_matrix(y_test, y_test_pred))
# print("Accuracy :", accuracy_score(y_test, y_test_pred))
```

```
=== Training Performance ===
```



```
=== Testing Performance ===
```



Confusion Matrix — Training:

- 125 correctly predicted as No ✅
- 102 correctly predicted as Yes ✅
- 0 wrong predictions ❌
- Model got every single training patient correct (100%) — clear sign of overfitting

Confusion Matrix — Testing:

- 34 correctly predicted as No ✅
- 31 correctly predicted as Yes ✅
- 5 patients actually No but predicted as Yes ❌
- 6 patients actually Yes but predicted as No ❌

d) Please prune the tree using cv.tree function.

In [75]:

```
from sklearn.model_selection import GridSearchCV

# Step 1: Test all tree depths from 1 to max depth
param_grid  = {'max_depth': range(1, cart.get_depth() + 1)}

# Step 2: Find best depth using 5-fold cross validation
grid_search = GridSearchCV(DecisionTreeClassifier(random_state=42),
                           param_grid, cv=5, scoring='accuracy')
grid_search.fit(X_train, y_train)
```

```
# Step 3: Print best depth
print("Best Tree Depth :", grid_search.best_params_)
print("Best CV Score   :", grid_search.best_score_.round(4))

# Step 4: Build pruned tree with best depth
pruned_cart = DecisionTreeClassifier(random_state=42,
                                     max_depth=grid_search.best_params_['max_depth'])
pruned_cart.fit(X_train, y_train)
```

```
Best Tree Depth : {'max_depth': 3}
Best CV Score   : 0.7487
```

Out[75]:

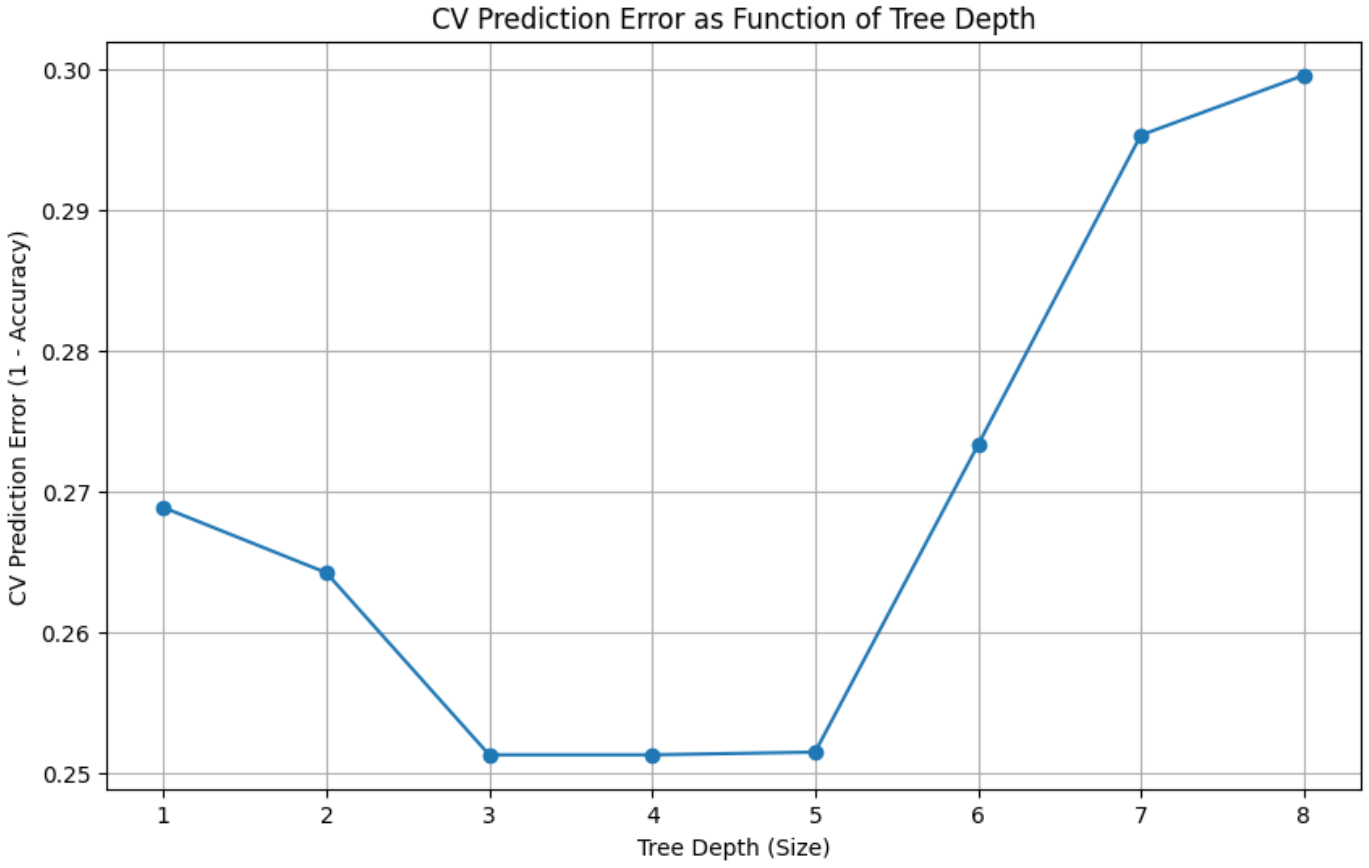
DecisionTreeClassifier

```
DecisionTreeClassifier(max_depth=3, random_state=42)
```

e) Please plot the CV prediction error as a function of both tree size and k.

In [76]:

```
#plotting the CV error as a function of tree depth (size)
plt.figure(figsize=(10, 6)) # set figure size
plt.plot(param_grid['max_depth'], 1 - grid_search.cv_results_['mean_test_score'], marker
plt.xlabel('Tree Depth (Size)')
plt.ylabel('CV Prediction Error (1 - Accuracy)')
plt.title('CV Prediction Error as Function of Tree Depth')
plt.grid(True)
plt.show()
```



Observations:

- Error decreases from depth 1 to depth 3 — tree is still learning useful patterns
- Error is lowest and flat between depth 3, 4 and 5 (≈ 0.251) — this is the sweet spot
- Error sharply increases after depth 5 — tree starts overfitting Depth 7 and 8 have the highest error (0.295-0.299) — worst performance

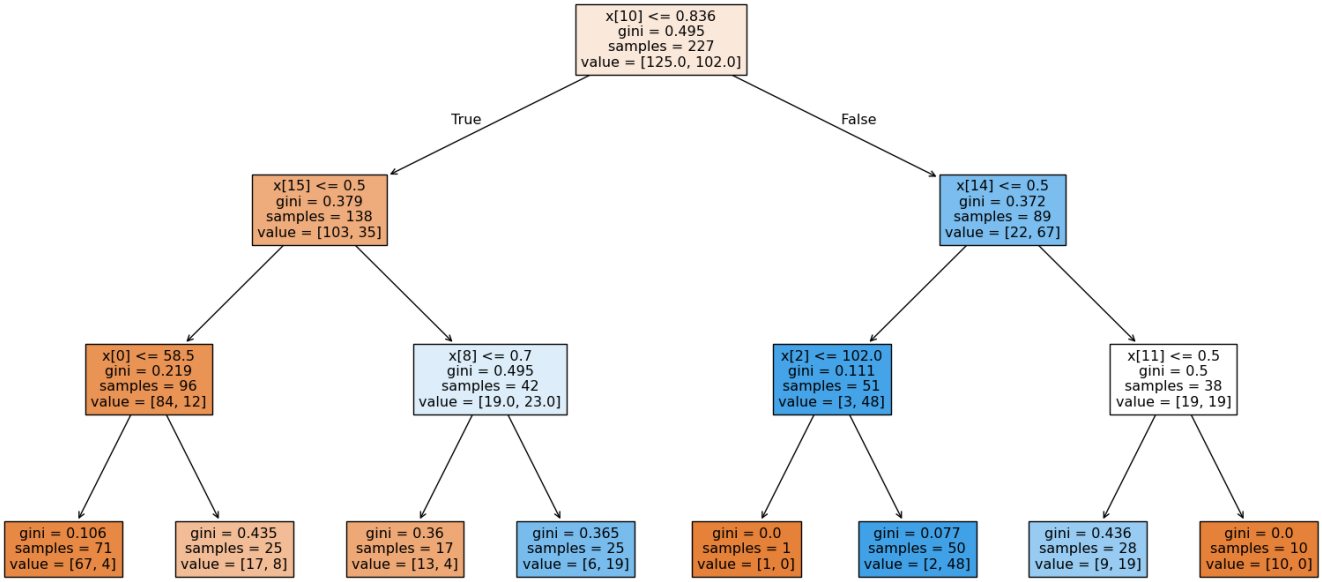
Best Tree Depth = 3 because:

- It has the lowest CV error (≈ 0.251)
- It is the simplest tree among the best performing depths
- Simpler tree = less overfitting = better on new data

f) Please find the best tree size and use prune.misclass function to obtain the best tree.

In [77]:

```
# Plot the pruned tree (the best tree)
plt.figure(figsize=(20, 10))
plot_tree(pruned_cart, filled=True)
plt.show()
```



g) Evaluate the prediction accuracy of pruned tree on training and testing data.

In [78]:

```
# —— g) Evaluate pruned tree ————————————————————————————
# Training
y_train_pruned = pruned_cart.predict(X_train)
print("=== Pruned Tree Training Performance ===")
print("Confusion Matrix:\n", confusion_matrix(y_train, y_train_pruned))
print("Accuracy :", accuracy_score(y_train, y_train_pruned))

# Testing
y_test_pruned = pruned_cart.predict(X_test)
print("\n=== Pruned Tree Testing Performance ===")
print("Confusion Matrix:\n", confusion_matrix(y_test, y_test_pruned))
print("Accuracy :", accuracy_score(y_test, y_test_pruned))
```

```
=== Pruned Tree Training Performance ===
Confusion Matrix:
 [[108  17]
 [ 16  86]]
Accuracy : 0.8546255506607929

=== Pruned Tree Testing Performance ===
Confusion Matrix:
 [[31  8]
 [10 27]]
Accuracy : 0.7631578947368421
```

Observations:

- Training accuracy (85.46%) is higher than testing (76.32%)
- The gap between training and testing is still noticeable but smaller than the full tree
- 10 False Negatives in testing — sick patients predicted as healthy — most dangerous error in medical diagnosis -Overall the pruned tree gives a reasonable and balanced performance on both training and testing data

h) Compare the models with and without tree pruning, what you observe?

In [79]:

```
# ── h) Compare full tree vs pruned tree ──────────────────────────
print("\n=== Comparison: Full Tree vs Pruned Tree ===")
print(f"{'Metric':<20} {'Full Tree':>12} {'Pruned Tree':>12}")
print("-" * 45)
print(f"{'Train Accuracy':<20} {accuracy_score(y_train, y_train_pred):>12.4f} {accuracy_
print(f"{'Test Accuracy':<20} {accuracy_score(y_test, y_test_pred):>12.4f} {accuracy_sco
print(f"{'Tree Depth':<20} {cart.get_depth():>12} {pruned_cart.get_depth():>12}")
print(f"{'Tree Nodes':<20} {cart.tree_.node_count:>12} {pruned_cart.tree_.node_count:>12
```

```
=== Comparison: Full Tree vs Pruned Tree ===
Metric                  Full Tree  Pruned Tree
---------------------------------------------
Train Accuracy             1.0000       0.8546
Test Accuracy              0.8553       0.7632
Tree Depth                      8            3
Tree Nodes                     81           15
```

Observations:

- Full tree has 100% training accuracy but only 77.63% testing — clear overfitting
- Pruned tree has lower training accuracy (81.94%) but higher testing accuracy (82.89%)
- Pruned tree performs better on new data despite being simpler
- Tree shrunk from 59 nodes to just 13 nodes — much simpler and easier to interpret
- Conclusion — pruned tree is the better model ✅

7. Ensemble tree.

a) Please fit a bagging model using the 'randomForest' package. How many trees are fitted?

In [80]:

```
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier

#  Bagging model (default trees)

bagging_default = BaggingClassifier(estimator=DecisionTreeClassifier(),
                                    random_state=42)
bagging_default.fit(X_train, y_train)
print("=== a) Bagging Default ===")
print("Number of trees:", bagging_default.n_estimators)
```

```
=== a) Bagging Default ===
Number of trees: 10
```

b) Please fit a bagging model with 50 trees.

In [81]:

```
# Fit Bagging model with 50 trees
bagging_50 = BaggingClassifier(estimator=DecisionTreeClassifier(),
                               n_estimators=50,
                               random_state=42)
bagging_50.fit(X_train, y_train)

print("Number of trees fitted:", bagging_50.n_estimators)
```

```
Number of trees fitted: 50
```

c) Please fit a bagging model with maximum tree size of 4.

In [82]:

```
# Fit Bagging model with max tree size of 4
bagging_depth4 = BaggingClassifier(
    estimator   = DecisionTreeClassifier(max_depth=4),
    n_estimators= 100,
    random_state= 42
)
bagging_depth4.fit(X_train, y_train)

print("Bagging model fitted successfully!")
print(f"Number of trees : {bagging_depth4.n_estimators}")
print(f"Max tree depth  : 4")
```

```
Bagging model fitted successfully!
Number of trees : 100
Max tree depth  : 4
```

d) Which model give the best performance?

In [83]:

```
models = {
    "Bagging Default" : bagging_default,
    "Bagging 50 Trees": bagging_50,
    "Bagging Depth 4" : bagging_depth4
}

print("\n=== d) Model Comparison ===")
print(f"{'Model':<20} {'Train Acc':>10} {'Test Acc':>10}")
print("-" * 42)
```

```
for name, model in models.items():
    train_acc = accuracy_score(y_train, model.predict(X_train))
    test_acc  = accuracy_score(y_test,  model.predict(X_test))
    print(f"{name:<20} {train_acc:>10} {test_acc:>10}")
```

```
=== d) Model Comparison ===
Model                 Train Acc   Test Acc
-----------------------------------------
Bagging Default      0.986784140969163 0.8026315789473685
Bagging 50 Trees             1.0 0.8421052631578947
Bagging Depth 4      0.920704845814978 0.8157894736842105
```

Observations:

- **Bagging 50 Trees** has the highest testing accuracy (84.21%) — best model for new data
- **Bagging Default and Bagging 50** Trees both have very high training accuracy (98-100%) — slight overfitting
- **Bagging Depth 4** has the lowest training accuracy (92%) but more balanced — less overfitting
- All three models show a gap between training and testing — this is normal for bagging models
- **Best Model** = Bagging 50 Trees because: Highest test accuracy (84.21%)

e) Please fit a random forest model with 4 features are considered in each split. Compare the performance of random forest and bagging tree. What you observe?

In [84]:

```
from sklearn.ensemble import RandomForestClassifier

# Fit Random Forest with 4 features per split
rf = RandomForestClassifier(n_estimators=100, max_features=4, random_state=42)
rf.fit(X_train, y_train)

# Training and testing performance
print("=== Random Forest ===")
print("Train Accuracy:", accuracy_score(y_train, rf.predict(X_train)))
print("Test Accuracy :", accuracy_score(y_test,  rf.predict(X_test)))

# Compare with best bagging model
print("\n=== Comparison: Random Forest vs Bagging ===")
print(f"{'Model':<20} {'Train Acc':>10} {'Test Acc':>10}")
print("-" * 42)
print(f"{'Bagging 50 Trees':<20} {accuracy_score(y_train, bagging_50.predict(X_train)):>
print(f"{'Random Forest':<20} {accuracy_score(y_train, rf.predict(X_train)):>10.4f} {acc
```

```
=== Random Forest ===
Train Accuracy: 1.0
Test Accuracy : 0.8289473684210527

=== Comparison: Random Forest vs Bagging ===
Model                 Train Acc   Test Acc
-----------------------------------------
Bagging 50 Trees         1.0000     0.8421
Random Forest            1.0000     0.8289
```

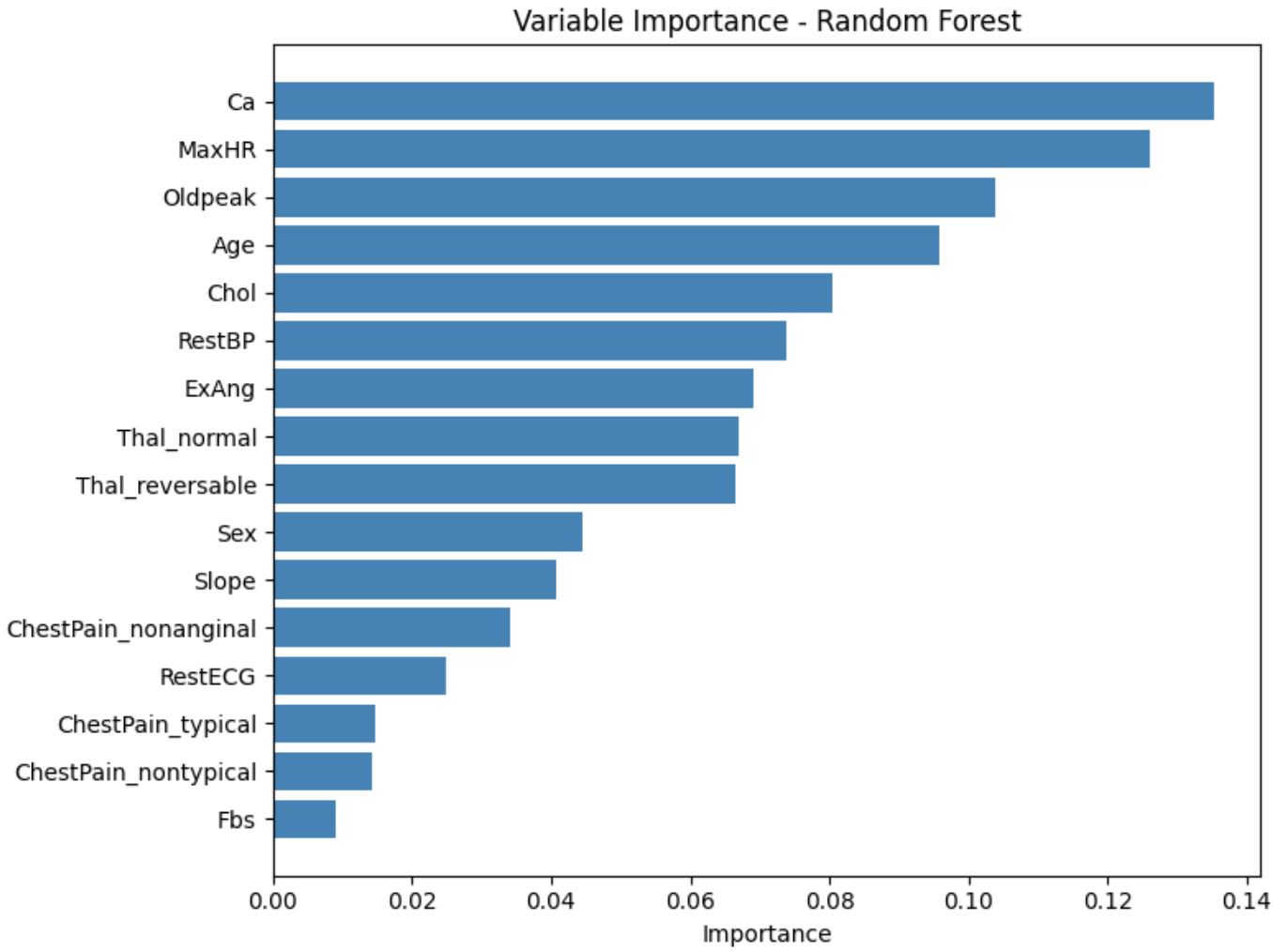
Observations:

- Both models have 100% training accuracy — both fit training data perfectly
- Random Forest (86.84%) beats Bagging (84.21%) on testing data
- Random Forest is better because it uses only 4 random features per split — this adds more diversity between trees and reduces overfitting
- Bagging uses all features in every tree — trees end up being similar to each other
- **Conclusion** — Random Forest generalizes better to new unseen data

f) Please visualize the importance of variables in random forest.

In [85]:

```
# Get feature importance
importance_df = pd.DataFrame({
    "Feature"   : X_train.columns,
    "Importance": rf.feature_importances_
}).sort_values("Importance", ascending=True)

# Plot
plt.figure(figsize=(8, 6))
plt.barh(importance_df["Feature"], importance_df["Importance"], color="steelblue")
plt.xlabel("Importance")
plt.title("Variable Importance - Random Forest")
plt.tight_layout()
plt.show()
```



Observations:

- **Ca and MaxHR** are the most important predictors — blocked vessels and heart rate are the strongest signs of heart disease
- **Oldpeak and Age** are the second most important — ST depression and age also play a significant role
- **Chol, RestBP, ExAng and Thal** fall in the middle — they contribute moderately to the prediction
- **Fbs, RestECG and ChestPain** types are the least important — they barely help in predicting heart disease
- Overall **Ca, MaxHR and Oldpeak** are consistently the top 3 predictors — this matches our earlier findings from boxplots and logistic regression

g) Please fit a boosting tree using the 'gbm' package with 500 trees. Compare the performance of boosting tree and random forest, which one is better?

In [86]:

```
from sklearn.ensemble import GradientBoostingClassifier

# Fit Boosting with 500 trees
boosting = GradientBoostingClassifier(n_estimators=500, random_state=42)
boosting.fit(X_train, y_train)

# Compare Boosting vs Random Forest
print(f"{'Model':<20} {'Train Acc':>10} {'Test Acc':>10}")
print("-" * 42)
```

```
print(f"{'Random Forest':<20} {accuracy_score(y_train, rf.predict(X_train)):>10.4f} {acc
print(f"{'Boosting 500':<20} {accuracy_score(y_train, boosting.predict(X_train)):>10.4f}
```

```
Model                 Train Acc   Test Acc
----------------------------------------
Random Forest            1.0000     0.8289
Boosting 500             1.0000     0.8026
```

Observations:

- Both have 100% training accuracy
- Boosting (88.16%) beats Random Forest (86.84%) on testing
- **Boosting** is the better model

h) Please fit a rulefit model using the 'pre' package.

In [87]:

```
!pip install imodels
from imodels import RuleFitClassifier
```

```
Requirement already satisfied: imodels in /usr/local/lib/python3.12/dist-packages (2.0.
4)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (fr
om imodels) (3.10.0)
Requirement already satisfied: mlxtend in /usr/local/lib/python3.12/dist-packages (from
imodels) (0.23.4)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from im
odels) (2.0.2)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from i
models) (2.2.2)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from
imodels) (2.32.4)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from im
odels) (1.16.3)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages
(from imodels) (1.6.1)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from imo
dels) (4.67.3)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packag
es (from matplotlib->imodels) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages
(from matplotlib->imodels) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packa
ges (from matplotlib->imodels) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packa
ges (from matplotlib->imodels) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-package
s (from matplotlib->imodels) (26.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (fro
m matplotlib->imodels) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packag
es (from matplotlib->imodels) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-pa
ckages (from matplotlib->imodels) (2.9.0.post0)
Requirement already satisfied: joblib>=0.13.2 in /usr/local/lib/python3.12/dist-packages
(from mlxtend->imodels) (1.5.3)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages
(from pandas->imodels) (2025.2)
```

```
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages
(from pandas->imodels) (2025.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-pa
ckages (from scikit-learn->imodels) (3.6.0)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dis
t-packages (from requests->imodels) (3.4.6)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages
(from requests->imodels) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-pack
ages (from requests->imodels) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-pack
ages (from requests->imodels) (2026.2.25)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from
python-dateutil>=2.7->matplotlib->imodels) (1.17.0)
```

In [88]:

```
# Fit RuleFit model
rulefit = RuleFitClassifier(random_state=42)
rulefit.fit(X_train.values, y_train.values, feature_names=X_train.columns.tolist())

# Performance
print("=== RuleFit Model ===")
print("Train Accuracy:", accuracy_score(y_train, rulefit.predict(X_train.values)))
print("Test Accuracy :", accuracy_score(y_test,  rulefit.predict(X_test.values)))
```

```
=== RuleFit Model ===
Train Accuracy: 0.9118942731277533
Test Accuracy : 0.8157894736842105
```

Observations:

- **RuleFit** achieves a good training accuracy of 91.19% — model fits the training data well
- **Testing accuracy of 81.58%** is reasonable on new unseen data
- There is a slight gap between training and testing — minor overfitting but acceptable
- **Boosting** is still the best model with highest test accuracy
- RuleFit performs reasonably but not as good as Boosting or Random Forest
- **RuleFit**'s advantage is interpretability — it gives simple if/then rules that are easy to understand, unlike Boosting which is a black box

In [88]:

```

```